MACHINE LEARNING REPORT

Jamie Smith
Department of Computer Science
Durham University
Durham, DH1 3LE
dtnx88@durham.ac.uk

December 15, 2025

ABSTRACT

EEG-based image classification presents a significant challenge for traditional machine learning algorithms. Since each person has a unique way of thinking, a machine learning model for this task typically needs to be trained on a large source of labelled data from each new person before it is able to make accurate predictions using their EEG signals. I address this problem by implementing a hybrid transfer/ensemble learning paradigm. Basic logistic regression classifiers are each trained on image and text data as well as the EEG signals from one of a range of source subjects, and I consider multiple ensemble approaches to combine these classifiers and adapt the combined model to an unseen target. A baseline parameter-averaged logistic regression model is compared with a custom class-weighted ensemble classifier, and adapted to the target using few-shot calibration. Experiments performed on the ThingsEEG dataset show a significant improvement in accuracy (68% vs 35%) under the custom ensemble, confirming improved robustness under domain shift.

1 Introduction

Traditional supervised learning classification algorithms, such as logistic regression or support vector machines, require a large volume of data to be able to make accurate predictions. This problem is magnified in the task of EEG-based image classification, where a model trained with any number of samples on one individual subject may not be able to make accurate predictions from the data of a different subject. This is because their brains can produce vastly different EEG signals when looking at the same objects. The naive way of rectifying this is to train a new model for each new subject, only using data from this subject. However, in practice, it is difficult and time-consuming to obtain sufficient labelled data from every new subject to produce a robust model.

To rectify this, more advanced paradigms such as few or zero-shot learning have been introduced, which leverage the power of deep learning techniques to be able to make accurate predictions with a very small training set. Although these techniques significantly outperform traditional methods, they come with their disadvantages, chiefly the complexity and computational cost of building robust models using these paradigms. As a result of these drawbacks, I decided to create a middle-ground solution by implementing a transfer learning paradigm using traditional classification models. This aims to take advantage of the inter-subject variability present in a large labelled dataset of EEG, image and text data to produce a generalised model that can then be fine-tuned in a few-shot fashion to an unseen target subject. This approach enables models to utilise preexisting data from a range of different subjects, only requiring a small amount of fine-tuning data from a new subject to be able to make accurate predictions for this new subject, without the computational cost that comes with implementing deep learning models.

In this report I document my exploration of the dataset that I worked with and how that influenced my decision to use a transfer learning approach. I then proceed to explain how my model was developed, starting from a simple baseline and then going on to implement a custom ensemble model to improve accuracy, with the final section of the report containing the analysis of the results from this improvement.

2 Data and Paradigm

The data used in the process of training, validating, and testing my model came from the "seen" portion of the BraVL ThingsEEG dataset (Gifford et al. [2022]). The data in this dataset was obtained by recording the brain EEG data for 10 subjects while they were being shown a series of 16540 images of 1654 different objects, with 10 different images for each object class. The same images and image classes were used across the 10 subjects. This EEG data, along with the image itself and some textual data relating to the image were transformed into features X and labels Y. The resulting dataset is of the form:

$$X_{brain} \in \mathbb{R}^{1020\times16540}, X_{image} \in \mathbb{R}^{100\times16540}, X_{text} \in \mathbb{R}^{512\times16540}, Y \in \mathbb{R}^{16540}$$

See Figure 2 for a diagram showing how the dataset is structured.
The same image samples were used for all 10 subjects to obtain the EEG data. As a result of this, the image and text features for each subject are the same, whereas the EEG features differ significantly between the subjects (see Figure 3). This variation in subject "personality" is further highlighted by Figure 1, revealing strong inter-subject domain shift which could form the basis for a transfer learning paradigm.

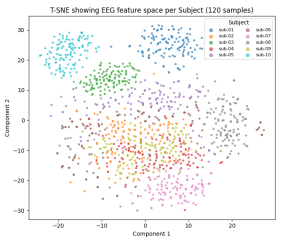


Figure 1: T-SNE plot showing the clusters formed by each subject's EEG data

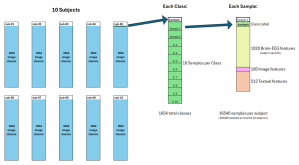


Figure 2: Visualisation of how the dataset is set out.
Each of the 10 subjects has the same 1654 classes and samples, but the values of the EEG features differ across subjects.

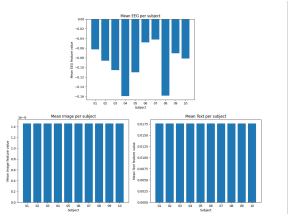


Figure 3: Graphs showing how the average values of the Image, Text and EEG features vary between the subjects

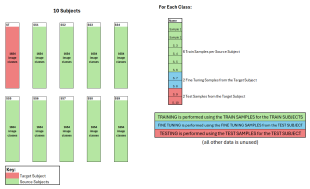


Figure 4: Train/Fine-Tuning/Test Split. The Train samples in the Test subject are unused and vice versa.

I decided to implement a hybrid transfer learning paradigm which draws inspiration from ensemble learning techniques to consider each of the subjects as independent entities, while using their combined knowledge as a general base upon which a model fine-tuned specifically to an unseen target subject can be built. The aim of this model is then to correctly classify unseen samples from the target subject. This paradigm allows for a personalised model to be built when limited data is available for the target subject, but there is ample available data for the same problem but from unrelated subjects.

Data from all three modalities seen in the dataset is used. One subject is designated as the Test Subject, *ST*, with the rest designated as Training Subjects, $SS_{1...9}$, with the data split as shown in Figure 4. The samples for each class were also separated between training, fine-tuning and testing to prevent data leakage, since the image and textual features for each sample are the same across the 10 subjects. In the context of transfer learning, subjects $SS_{1...9}$ form the Source Domain, with *ST* being the Target Domain.

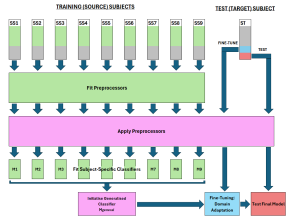


Figure 5: Paradigm Workflow visualisation

The features across the three modalities are combined with an early fusion approach using dimensionality reduction, producing a single feature vector for each sample. Once this preprocessing has occurred, for each training subject SS_i, an instance of a supervised classifier M_i is trained on the training data from that subject, with each of these models producing a matrix of coefficients W_i, capturing subject-specific patterns for SS_i. These coefficient matrices are aggregated to create a generalised, subject-agnostic classifier $M_{general}$, in an ensemble-like fashion. This classifier is then adapted to *ST* by fine-tuning it using few-shot calibration with previously unseen samples from *ST*. This allows the model to adapt the general knowledge learned from $SS_{1...9}$ to patterns specific to *ST*. Finally, the ability of the model to adapt to the target domain is determined by testing the model using *ST*. A visualisation of the full workflow can be found in Figure 5.

During model development and refinement, Leave One Subject Out cross validation was employed to tune

hyperparameters and optimise the model for a general unseen target subject, to prevent overfitting to the test subject itself. For this process, one subject SS_v was iteratively held out from training to act as a target subject for validation. This process was repeated for each different SS_v, without involving *ST*, to maximise the average CV accuracy.

3 Model Development

I began model development by first implementing a simple baseline model using scikit-learn (Pedregosa et al. [2011]), then implemented a custom ensemble model to improve accuracy when transferring knowledge to the target. For both the baseline and improved models, the input space is defined as the set of all possible 1632-dimensional feature vectors $X = \mathbb{R}^{1632}$, with a single data sample $x \in X$ defined as $x = [x_1, x_2, ..., x_{1632}]$. The output space is defined as $Y = \{c_1, c_2, ..., c_{1654}\}$, i.e. the set of all possible class labels defined in the dataset. Since this is a classification task, the overall objective of the model is to find an optimal hypothesis function $h : X \rightarrow Y$ which minimises prediction errors when making predictions predictions $\hat{y}$ for unseen inputs $\hat{x}$ from the target subject *ST*.

My initial baseline model contains three distinct components: the preprocessors, the individual classifiers for each training subject, and the final generalised model, fine-tuned to the target.

To preprocess the data, I fitted a scaler to the training data, which calculates the normalised value z for each sample x by calculating $z = \frac{x-\mu}{\sigma}$, where μ and σ are the mean and standard deviation of the training set respectively. To reduce the computational cost of training the model, unsupervised agglomerative clustering was used to group together correlated features to reduce the dimensionality of the feature space $X \in \mathbb{R}^{1632} \rightarrow X' \in \mathbb{R}^f$, where f is a hyperparameter to be determined. These transformations were implemented using the StandardScaler and FeatureAgglomeration classes respectively, defined in scikit-learn. The preprocessors are subsequently applied to the fine-tuning and test data.

To obtain subject-specific patterns for each of the training subjects $SS_{1...9}$, a basic Logistic Regression (LR) model with level 2 penalty was trained on each SS_i. Each of these LR models M_i minimises the cross-entropy loss function L_i for SS_i, defined as

$$L_i = -\frac{1}{|Train_i|} \sum_{x \in Train_i} \sum_{c=1}^{N} y_c ln(\hat{p}_c) + \frac{1}{2}||W||_2^2$$

for that specific subject's training data, where $Train_i$ is the training set for subject SS_i with $|Train_i|$ being the number of samples in the training set for SS_i, N is the total number of classes, $\hat{p}_c$ is the predicted probability for class c, and $y_c = 1$ if c is the true class, 0 otherwise. $\frac{1}{2}||W||_2^2$ is the level 2 penalty term, as defined by scikit-learn - these LR models were implemented using scikit-learn.

Fitting each of these models M_i produces a matrix of coefficients $W_i \in \mathbb{R}^{N \times f}$ encoding the subject-specific patterns for each SS_i, found in training. As an initial baseline, a simple parameter averaging approach was employed to produce a basic generalised model $M_{general}$ by initialising a warm-start LR classifier with coefficients

$$W_{general} = \frac{1}{n_{SS}} \sum_{i=1}^{n_{SS}} W_i$$

where n_{SS} is the number of training subjects. This model was then fine-tuned using two unseen samples from each class $X_{ft} \in \mathbb{R}^{f \times 2N}$, $y_{ft} \in \mathbb{R}^{2N}$ from ST, where f is the number of features in the preprocessed data and N is the number of classes. This enables the model to be tuned to target-specific patterns while only needing a small amount of data from ST. Using unseen samples in this step prevented overfitting to the specific image and text features present in the training data. This final model M_t is then evaluated on different unseen test samples from ST.

Although this parameter-averaging approach was reasonably successful, given its simplicity, it was clear that a better method of ensembling the subject-specific models was needed. The original approach could cause any outlier training subjects to severely impact the resulting parameters, and the averaged model is also prone to overfitting to the specific samples seen in fine-tuning due to the few-shot nature of the calibration. Furthermore, parameter averaging blurs all the different subject-specific trends and patterns seen in training. All of these issues could lead to degradation of the performance of the final model, as it does not take full advantage of the multi-source nature of the data collected in the

intermediate models M_i.

To rectify this, I implemented a custom class-weighted voting classifier (CWVC), which maintains the separation between the estimators M_i, built on top of the scikit-learn API (Buitinck et al. [2013]) . This ensemble classifier takes in as input the n base estimators M_i, and creates a generalised model $M_{general}$ using basic majority voting. The class-based voting weights $\mathbf{W_i}$ for each M_i are then fine-tuned using the same two-shot calibration data X_{ft} from ST as in the baseline.
For each M_i, the predicted classes $\hat{y}_i$ for X_{ft} are compared with y_{ft} to calculate the binary F1 score of this estimator for each class c_j as follows:

$$F1_{i,j} = \frac{2 \times TP_{i,j}}{2 \times TP_{i,j} + FP_{i,j} + FN_{i,j}}$$

Where $TP_{i,j}, FP_{i,j}, FN_{i,j}$ represent the true positives, false positives and false negatives respectively for class c_j in $\hat{y}_i$. The overall multiclass accuracy for each estimator a_i is also calculated, and these are normalised to $[0, 1]$. Laplace smoothing is applied as follows to each $F1_{i,j}$ to improve stability by removing zero values from F1 scores:

$$F1'_{i,j} = \frac{F1_{i,j} + \alpha}{1 + \alpha}$$

Here, α is a hyperparameter. The class based weight $\mathbf{W}_{i,j}$ for each class c_j for estimator M_i are then defined as $\mathbf{W}_{i,j} = F1'_{i,j} \times a_i$, and normalised such that for each estimator M_i:

$$\sum_j \mathbf{W}_{i,j} = 1$$

When making predictions on a new sample $x_{test} \in \mathbb{R}^f$, the weighted probability vector $\mathbf{W'_i}$ for each M_i is calculated as follows:

$$\mathbf{W'_i} = \hat{\mathbf{p}}_\mathbf{i} \cdot \mathbf{W_i}$$

Where $\hat{p}_i \in \mathbb{R}^N$ is the predicted probability vector for x_{test} from M_i, and W_i is the vector of class weights for estimator M_i as calculated above. The overall probability vector $\hat{\mathbf{p}}$ is defined as:

$$\hat{\mathbf{p}} = \sum_i \mathbf{W'_i}$$

The final class predicted by the CWVC is defined as $\hat{y}_{test} = argmax(\hat{\mathbf{p}})$. The overall aim of this classifier is to weight each M_i based on how similar SS_i "thinks" compared to the target when faced with images of each class. This maintains and utilises the separation of the training subjects to compare ST with each SS_i individually on a class-by-class basis instead of producing a single average model and adapting it to ST.

The improved model contains two hyperparameters: f - the number of resulting features per sample after the dimensionality reduction, and α, the Laplace smoothing parameter used to smooth the F1 scores before calculating the class weights in the CWVC. To prevent overfitting to a specific target, Leave One Subject Out cross-validation (LOSO-CV) was employed to tune these hyperparameters. In this process, eight variations of the model are trained, with one training subject held out as a validation subject SV each time. The optimum values of the hyperparameters were determined by maximising the average accuracy across these eight variations when working with a subset of 100 classes. Figure 6 shows how the LOSO-CV accuracy varied depending on the values of f and α, with the optimal values determined to be $f = 565, \alpha = 1.0$.

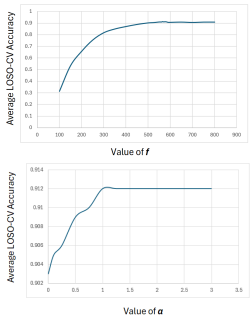


Figure 6: Graphs showing how accuracy varies with f and α

4 Results Analysis

The improved CWVC approach performs significantly better than the baseline parameter averaging approach. Figure 7 displays the basic metrics for both models, as well as the average of each metric across the subject-specific LR classifiers M_i, when working with all 1654 classes in the dataset. This shows that the individual M_i classifiers actually performed better than the fine-tuned generalised model. This is most likely caused by cross-subject noise created as a result of averaging across subjects with vastly different EEG data. The improved model mitigates this noise by considering all the training subjects separately, and assigning low weights to those vastly different to the target.

Figure 8 shows the precision-recall (PR) curves for both models. These clearly show that the improved model is significantly more precise than the baseline for almost all recall values, meaning that the improved model makes fewer false positives on average. Furthermore, the area under the improved PR curve, representing the average precision (AP) score, is considerably greater than that of the baseline. Both of these facts emphasise that the improved model is much more robust and performs better than the baseline. This is reinforced by Figure 9, which shows that the improved model reduces the number of classes with a near-zero AP, while increasing the proportion of classes achieving a high AP, confirming that the improvements made to the baseline have improved robustness across a broad range of classes.

Metric	Average M_i [1]	Baseline	Improved
Accuracy	0.510	0.347	0.679
Precision	0.534	0.333	0.674
Recall	0.508	0.347	0.679
F1	0.492	0.317	0.650
Macro AP	0.629	0.405	0.770
Micro AP	0.466	0.257	0.716

Figure 7: Comparison of various metrics between the baseline and improved models, as well as the average metrics across the subject-specific classifiers M_i.

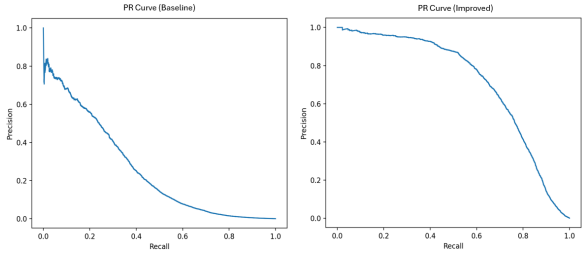


Figure 8: Precision-Recall curves for the baseline and improved models

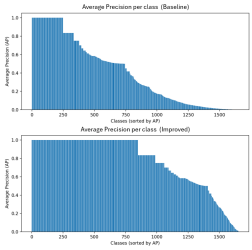


Figure 9: Histograms showing the AP per class for both models (sorted)

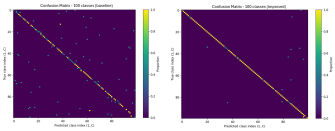


Figure 10: Confusion matrices for both models when working with 100 classes

Figure 10 shows confusion matrix heatmaps produced by both models when working with 100 classes from the dataset (for ease of visualisation). These clearly show that the predictions made by the improved model are much more strongly aligned with the correct classes, with fewer failure cases overall. However, this also shows that the improved model is far from perfect - even with a small subset of classes there are still some misclassifications. This could be a result of the test samples for some classes differing considerably from the samples seen in training and fine-tuning, which is a problem that can only be rectified using a larger dataset.

Another cause of failure cases is that some classes in the dataset are of conceptually similar objects and the model is unable to differentiate between them. For instance Figure 10 shows that the improved model misclassified both test samples of class index 99 as members of class index 97, likely since class 97 represents "bed" while class 99 represents "bed-post", with very similar images. This similarity in concept and images causes the features across all three modalities for these classes to blend together, leading to the model being unable to differentiate between these classes. This issue is magnified when all 1654 classes are considered, and is likely the main cause of inaccuracies for this improved model. Fully rectifying this issue would likely require the use of deep neural networks or reinforcement learning, which would introduce unwanted complexity and computational cost.

References

Alessandro Gifford, Kshitij Dwivedi, Gemma Roig, and Radoslaw M. Cichy. A large and rich EEG dataset for modeling human visual object recognition. *bioRxiv*, 2022. doi:10.1101/2022.03.15.484473.

F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

Lars Buitinck, Gilles Louppe, Mathieu Blondel, Fabian Pedregosa, Andreas Mueller, Olivier Grisel, Vlad Niculae, Peter Prettenhofer, Alexandre Gramfort, Jaques Grobler, Robert Layton, Jake VanderPlas, Arnaud Joly, Brian Holt, and Gaël Varoquaux. API design for machine learning software: experiences from the scikit-learn project. In *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*, pages 108–122, 2013.

[1] The non-tuned value of $f = 450$ was used in this case (same as baseline)